SUPPLY MANAGEMENT SYSTEM — Functional Specification Document

FSD Addendum 42 — Chart of Accounts Configuration, Opening Balances & Financial Integration

Property	Value
Document ID	SMS-FSD-ADD-042
Version	1.0
Date	2026-10-12
Status	Draft — Pending Approval
Author	System Architect
Classification	Internal — Development
Supersedes	None — new module under `finance` schema
Architecture	.NET 8 / EF Core 8 / SQL Server 2022 / React 18
Change Items	10 new features
Impact	12 migrations, 10 new entities, 1 modified entity, 2 new lookups, 7 new UI panels, 1 state machine
Depends On	Addendum 37 (Module Registry — Finance module activation)
Deferred to Addendum 43	General Ledger Journal Engine, Accounts Receivable Sub-Ledger, Accounts Payable Sub-Ledger, Bank Reconciliation, Multi-Currency Transactions, Tax Configuration & Reporting, Financial Statements Generator
Naming Convention	Finance entities use standard accounting terminology (ChartOfAccount, JournalEntry, FiscalPeriod). All entity names are singular. Field names use snake_case.

Table of Contents

1. Purpose & Scope
2. Chart of Accounts — Conceptual Overview
3. C1 — Finance Module Activation & Schema
4. C2 — CoA Template Registry

5. C3 — CoA Template Items
6. C4 — Tenant Chart of Accounts (4-Level Hierarchy)
7. C5 — Fiscal Periods
8. C6 — Accounting Defaults
9. C7 — CoA Setup Status & Module Gate
10. C8 — Opening Balances
11. C9 — CoA Setup Wizard & Guide
12. C10 — External Financial System Integration Hooks
13. Database Migrations
14. API Endpoints
15. UI Wireframes & Specifications
16. Business Rules
17. Test Scenarios
18. Development Phases

---

1. Purpose & Scope

This addendum establishes the financial foundation of SMS — the Chart of Accounts (CoA) configuration system that every other financial feature depends on. It defines the template library, the 4-level account hierarchy, fiscal period management, opening balance entry, and integration hooks for external financial systems (QuickBooks, Xero, ERPNext, etc.).

Before this addendum, SMS has no financial backbone. Modules such as Procurement (PO → GRN → Invoice) and Sales (Inquiry → Quotation → Order) capture commercial transactions but have nowhere to record their financial impact. This addendum creates the `finance` schema and the account structure that all future financial addenda will post against.

The design follows a **"configure first, operate later"** pattern: tenants must complete Chart of Accounts setup before any financial module becomes operational. This prevents orphaned transactions and ensures every debit/credit has a valid destination account from day one.

1.1 Design Principles

- **4-Level Fixed Hierarchy** — SMS enforces exactly four levels in the Chart of Accounts: Level 1 (Root Type — 5 fixed categories), Level 2 (Major Group), Level 3 (Sub-Group), Level 4 (Detail/Leaf Account). Only Level 4 accounts are postable. This strikes the balance between structural flexibility and reporting consistency across tenants.
- **Template-Driven Setup** — SMS ships with a library of pre-built CoA templates organized by country and business type (e.g., "United States — Manufacturing", "United Kingdom — Retail"). Tenants select a template as their starting point and customize from there, rather than building from scratch.

- **Download Before Upload** — Templates are downloadable as CSV and JSON so tenants can review, modify, and re-upload them. This supports offline editing, Excel-based review cycles, and migration from legacy systems.
- **User Upload of Custom Structures** — Tenants can upload their own CoA structure (CSV or JSON) that conforms to the 4-level schema, bypassing the template library entirely. This supports companies migrating from existing accounting systems.
- **Opening Balance as First Entry** — After CoA setup, tenants enter opening balances against each account head. The system records these as a special journal entry (type = OPENING_BALANCE) dated on the conversion date, using a temporary "Opening Balance Equity" account (3900) as the balancing plug.
- **Module Gate** — Financial modules (Invoicing, Payments, Bank Reconciliation, etc.) check `finance.CoaSetupStatuses` before allowing any operation. Until CoA setup is marked complete, those modules display a redirect to the CoA Setup Wizard.
- **Integration-Ready** — The CoA structure includes an `external_code` field on every account and a `finance.ExternalSystemMappings` table that maps SMS accounts to their counterparts in QuickBooks, Xero, ERPNext, Tally, and other external systems. Account sync direction (inbound, outbound, bidirectional) is configurable per mapping.
- **Multi-Tenant by Default** — Every entity carries `org_id` with EF Core `HasQueryFilter`. Templates owned by SMS (system templates) have `org_id = NULL`; tenant-customized copies have the tenant's `org_id`.
- **Adjacency List + Materialized Path** — Account hierarchy is stored with both a `parent_account_id` FK (for parent-child navigation) and a `materialized_path` string (for fast subtree queries via `LIKE '/1/11/%'`). Recursive CTEs are used for tree traversal.

1.2 Scope Summary

Change ID	Title	Type	Impact
C1	Finance Module Activation & Schema	Module Config	Activate Finance module in Module Catalog, create `finance` schema
C2	CoA Template Registry	New Entity	1 new table (CoaTemplates) with system-seeded templates
C3	CoA Template Items	New Entity	1 new table (CoaTemplateItems) — accounts within a template
C4	Tenant Chart of Accounts	New Entity + UI	1 new table (ChartOfAccounts), 4-level hierarchy, 2 new UI panels
C5	Fiscal Periods	New Entity + UI	2 new tables (FiscalYears, FiscalPeriods), 1 new UI panel
C6	Accounting Defaults	New Entity + UI	1 new table (AccountingDefaults), 1 new UI panel
C7	CoA Setup Status & Module Gate	New Entity + Logic	1 new table (CoaSetupStatuses), gate middleware
C8	Opening Balances	New Entity + UI	1 new table (OpeningBalanceEntries), 1 new UI panel
C9	CoA Setup Wizard & Guide	UI + Logic	Multi-step wizard, contextual help, progress tracking
C10	External Financial System Integration Hooks	New Entity	1 new table (ExternalSystemMappings), sync direction config

1.3 Relationship to Existing Entities

Existing Entity	Schema	Addendum 42 Interaction
core.ModuleCatalog	core	MODIFIED — Finance module row updated: is_available = 1
core.Organizations	core	REFERENCED — org_id FK on all finance entities
auth.Users	auth	REFERENCED — created_by / modified_by audit fields
lookups.Currencies	lookups	REFERENCED — CoaTemplates.currency_code, FiscalYears.reporting_currency_code
lookups.Countries	lookups	REFERENCED — CoaTemplates.country_code

1.4 Double-Entry Bookkeeping Context

SMS follows the universal double-entry bookkeeping system. Every financial transaction creates a journal entry with at least two lines where total debits equal total credits. The five root account types and their normal balances are:

Root Type	Normal Balance	Increases With	Decreases With	Examples
Asset	Debit	Debit	Credit	Cash, Bank, Inventory, Equipment, Accounts Receivable
Liability	Credit	Credit	Debit	Accounts Payable, Loans, Accrued Expenses
Equity	Credit	Credit	Debit	Owner's Capital, Retained Earnings, Opening Balance Equity
Revenue	Credit	Credit	Debit	Sales Revenue, Service Income, Interest Income
Expense	Debit	Debit	Credit	Salaries, Rent, Utilities, COGS, Depreciation

The accounting equation **Assets = Liabilities + Equity + (Revenue − Expenses)** must always hold. Opening balances, posted as journal entries on the conversion date, must satisfy this equation.

---

2. Chart of Accounts — Conceptual Overview

2.1 The 4-Level Hierarchy

SMS enforces exactly four levels. Level 1 is fixed and non-editable. Levels 2–4 are configured per tenant (from templates or manual/uploaded setup). Only Level 4 accounts accept postings.

```
Level 1 — Root Type (FIXED — system-defined, non-editable)
│
├── 1 Assets
├── 2 Liabilities
├── 3 Equity
├── 4 Revenue
└── 5 Expenses

Level 2 — Major Group (user-configurable)
│
├── 11 Current Assets
├── 12 Non-Current Assets
├── 21 Current Liabilities
├── 22 Long-Term Liabilities
├── 31 Owner's Equity
├── 41 Operating Revenue
├── 42 Other Income
├── 51 Cost of Goods Sold
├── 52 Operating Expenses
└── 53 Other Expenses

Level 3 — Sub-Group (user-configurable)
│
├── 111 Cash & Cash Equivalents
├── 112 Receivables
├── 113 Inventory
├── 121 Property, Plant & Equipment
├── 122 Intangible Assets
├── 211 Trade Payables
├── 511 Direct Material Costs
├── 521 Personnel Expenses
└── ...

Level 4 — Detail / Leaf Account (POSTABLE — user-configurable)
│
├── 1111 Petty Cash
├── 1112 Cash in Hand — Main
├── 1113 Cash in Hand — Branch
├── 1121 Accounts Receivable — Trade
├── 1122 Accounts Receivable — Staff Advances
├── 1131 Raw Materials Inventory
├── 1132 Finished Goods Inventory
├── 1211 Land
```

```
├── 1212 Buildings
├── 1213 Machinery & Equipment
├── 2111 Accounts Payable — Trade
├── 5111 Raw Material Purchases
├── 5211 Salaries & Wages
└── ...
```

2.2 Group vs. Leaf Account Rules

Property	Group Account (Levels 1–3)	Leaf Account (Level 4)
`is_group`	`true`	`false`
Accepts journal postings	No	Yes
Shows in trial balance detail	No (subtotal only)	Yes
Can have children	Yes	No
Can be deleted	Only if no children exist	Only if no journal entries posted
Appears in account selectors	No	Yes

2.3 Account Code Numbering Convention

SMS uses a numeric coding system where the position encodes the hierarchy:

Level	Digits	Range	Example
Level 1 — Root Type	1 digit	1–5	`1` = Assets
Level 2 — Major Group	2 digits	11–59	`11` = Current Assets
Level 3 — Sub-Group	3 digits	111–599	`111` = Cash & Cash Equivalents
Level 4 — Detail Account	4 digits	1111–5999	`1111` = Petty Cash

The first digit of any account code identifies its root type: 1=Asset, 2=Liability, 3=Equity, 4=Revenue, 5=Expense. This convention is enforced by the system during template creation and manual account entry.

2.4 Template-to-Tenant Flow

---

3. C1 — Finance Module Activation & Schema

3.1 Description

Register the Finance module in the Module Catalog and create the `finance` schema in SQL Server. This follows the same pattern established in Addendum 37 (Module Registry) and Addendum 41 (CRM schema).

3.2 Module Catalog Entry

Column	Value
module_code	`FINANCE`
module_name	`Financial Management`
schema_name	`finance`
description	`Chart of Accounts, General Ledger, Accounts Receivable, Accounts Payable, Banking, and Financial Reporting`
is_available	`1` (active from this addendum)
display_order	`20`
icon	`currency-dollar`
depends_on	`NULL` (Finance is a foundational module — no module dependencies)

3.3 Schema Creation

```sql
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'finance')
    EXEC('CREATE SCHEMA finance');
```

3.4 EF Core Migration

Migration `AddFinanceSchema` creates the schema and updates the Module Catalog seed:

```csharp
migrationBuilder.Sql("CREATE SCHEMA finance");

migrationBuilder.UpdateData(
    table: "ModuleCatalog",
    schema: "core",
    keyColumn: "module_code",
    keyValue: "FINANCE",
    column: "is_available",
    value: true);
```

---

4. C2 — CoA Template Registry

4.1 Description

A system-managed library of Chart of Accounts templates. Each template represents a complete account structure suited to a specific country and business type combination. Templates are read-only for tenants — they select one to apply but cannot modify the system copy. SMS ships with templates for major economies; new templates are added by system administrators via seed data.

4.2 Entity Definition — `finance.CoaTemplates`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
template_code	nvarchar(50)	NO	—	Unique code (e.g., `US_GENERAL`, `UK_RETAIL`, `DE_MANUFACTURING`)
template_name	nvarchar(200)	NO	—	Display name (e.g., "United States — General Business")
country_code	nvarchar(3)	YES	NULL	ISO 3166-1 alpha-3 country code. NULL = country-agnostic template
business_type	nvarchar(100)	YES	NULL	Business type label (e.g., "General", "Retail", "Manufacturing", "Services", "Non-Profit"). NULL = general-purpose
currency_code	nvarchar(3)	YES	NULL	Default currency for this template (ISO 4217). NULL = use tenant's base currency
description	nvarchar(1000)	YES	NULL	Explains when to use this template
account_count	int	NO	0	Cached count of items in CoaTemplateItems for this template
is_active	bit	NO	1	Soft-delete flag
version	nvarchar(20)	NO	`'1.0'`	Template version for tracking updates
org_id	int	YES	NULL	NULL = system template (global). Non-null = tenant-created custom template
created_at	datetime2(7)	NO	GETUTCDATE()	Record creation timestamp
created_by	int	YES	NULL	FK → auth.Users. NULL for system-seeded data
modified_at	datetime2(7)	YES	NULL	Last modification timestamp
modified_by	int	YES	NULL	FK → auth.Users

Indexes:

Index	Columns	Type	Notes
UX_CoaTemplates_Code_Org	template_code, org_id	Unique, Filtered (WHERE is_active = 1)	Same code can exist once per org (and once globally with org_id NULL)
IX_CoaTemplates_Country	country_code	Non-unique	Filter templates by country
IX_CoaTemplates_BusinessType	business_type	Non-unique	Filter templates by business type
IX_CoaTemplates_OrgId	org_id	Non-unique	Tenant filter

Constraints:

- `template_code` must match pattern `^[A-Z0-9_]{2,50}$`
- `country_code` must exist in `lookups.Countries` when non-null
- `currency_code` must exist in `lookups.Currencies` when non-null

4.3 System-Seeded Templates

SMS ships with the following initial templates. Additional templates are added via future seed migrations.

template_code	template_name	country_code	business_type
GLOBAL_GENERAL	Global — General Business	NULL	General
GLOBAL_SERVICES	Global — Professional Services	NULL	Services
GLOBAL_NONPROFIT	Global — Non-Profit Organization	NULL	Non-Profit
US_GENERAL	United States — General Business	USA	General
US_RETAIL	United States — Retail	USA	Retail
US_MANUFACTURING	United States — Manufacturing	USA	Manufacturing
US_SERVICES	United States — Professional Services	USA	Services
GB_GENERAL	United Kingdom — General Business	GBR	General
GB_RETAIL	United Kingdom — Retail	GBR	Retail
CA_GENERAL	Canada — General Business	CAN	General
AU_GENERAL	Australia — General Business	AUS	General
DE_GENERAL	Germany — General Business (SKR04)	DEU	General
AE_GENERAL	United Arab Emirates — General Business	ARE	General
SA_GENERAL	Saudi Arabia — General Business	SAU	General
IN_GENERAL	India — General Business	IND	General
PK_GENERAL	Pakistan — General Business	PAK	General

5. C3 — CoA Template Items

5.1 Description

Individual account entries within a template. Each item represents one row in the 4-level hierarchy. Template items are stored in the database (seeded via migration) and can be exported as CSV or JSON for tenant download.

5.2 Entity Definition — `finance.CoaTemplateItems`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
template_id	int	NO	—	FK → finance.CoaTemplates.id
account_code	nvarchar(20)	NO	—	Account code (e.g., "1111")
account_name	nvarchar(200)	NO	—	Account name (e.g., "Petty Cash")
parent_item_id	int	YES	NULL	FK → self (finance.CoaTemplateItems.id). NULL for Level 1 roots
level	tinyint	NO	—	1, 2, 3, or 4. Enforced CHECK (level BETWEEN 1 AND 4)
root_type	nvarchar(20)	NO	—	'Asset', 'Liability', 'Equity', 'Revenue', 'Expense'
is_group	bit	NO	—	true for Levels 1–3, false for Level 4
account_type	nvarchar(50)	YES	NULL	Sub-classification for Level 4: 'Cash', 'Bank', 'Receivable', 'Payable', 'Tax', 'COGS', 'Depreciation', 'Equity', etc.
description	nvarchar(500)	YES	NULL	Help text for this account
sort_order	int	NO	0	Display order within parent
is_default_account	bit	NO	0	If true, this account is auto-assigned in AccountingDefaults when template is applied
default_account_key	nvarchar(50)	YES	NULL	Which default this fulfills (e.g., 'DEFAULT_RECEIVABLE', 'DEFAULT_PAYABLE', 'DEFAULT_CASH'). Non-null only when is_default_account = 1

Indexes:

Index	Columns	Type	Notes
UX_CoaTemplateItems_Template_Code	template_id, account_code	Unique	No duplicate codes within a template
IX_CoaTemplateItems_Parent	parent_item_id	Non-unique	Tree traversal
IX_CoaTemplateItems_TemplateId	template_id	Non-unique	Template filter
IX_CoaTemplateItems_Level	template_id, level	Non-unique	Filter by level within template

Constraints:

- `level` CHECK: `level BETWEEN 1 AND 4`
- `root_type` CHECK: `root_type IN ('Asset', 'Liability', 'Equity', 'Revenue', 'Expense')`
- `is_group` must be `true` when `level IN (1, 2, 3)` and `false` when `level = 4`
- `parent_item_id` must be NULL when `level = 1`
- `parent_item_id` must reference an item with `level = current_level - 1` within the same template
- `account_type` CHECK: `account_type IN ('Cash', 'Bank', 'Receivable', 'Payable', 'Tax', 'CostOfGoodsSold', 'Revenue', 'Expense', 'FixedAsset', 'Depreciation', 'Equity', 'AccumulatedDepreciation', 'OtherCurrentAsset', 'OtherCurrentLiability', 'LongTermLiability', 'OtherIncome', 'OtherExpense', NULL)`

5.3 CSV Export Format

When a tenant downloads a template as CSV, the columns are:

```csv
```

```
account_code,account_name,parent_code,level,root_type,is_group,account_type,description
1,Assets,,1,Asset,true,,Root category for all asset accounts
11,Current Assets,1,2,Asset,true,,Assets expected to be converted to cash within one year
111,Cash & Cash Equivalents,11,3,Asset,true,,Liquid funds immediately available
1111,Petty Cash,111,4,Asset,false,Cash,Small cash fund for minor expenses
1112,Cash in Hand,111,4,Asset,false,Cash,Physical currency held on premises
1113,Cash at Bank – Operating,111,4,Asset,false,Bank,Primary operating bank account
112,Receivables,11,3,Asset,true,,Amounts owed to the business
1121,Accounts Receivable – Trade,112,4,Asset,false,Receivable,Customer balances for goods/ser
```

- `parent_code` references the `account_code` of the parent (not the id), making the CSV portable.
- Boolean `is_group` is exported as text `true` / `false`.
- Empty `account_type` for group accounts.

5.4 JSON Export Format

When a tenant downloads a template as JSON, the structure is a nested tree:

```json
```

```
{
  "template_code": "US_GENERAL",
  "template_name": "United States – General Business",
  "country_code": "USA",
  "currency_code": "USD",
  "version": "1.0",
  "accounts": [
    {
      "code": "1",
      "name": "Assets",
      "root_type": "Asset",
      "level": 1,
      "is_group": true,
      "children": [
        {
          "code": "11",
          "name": "Current Assets",
          "root_type": "Asset",
          "level": 2,
          "is_group": true,
          "children": [
            {
              "code": "111",
              "name": "Cash & Cash Equivalents",
              "root_type": "Asset",
              "level": 3,
              "is_group": true,
              "children": [
                {
                  "code": "1111",
                  "name": "Petty Cash",
                  "root_type": "Asset",
                  "level": 4,
                  "is_group": false,
                  "account_type": "Cash",
                  "description": "Small cash fund for minor expenses"
                },
                {
                  "code": "1112",
                  "name": "Cash in Hand",
                  "root_type": "Asset",
                  "level": 4,
                  "is_group": false,
```

```
                "account_type": "Cash"
              }
            ]
          }
        ]
      }
    ]
  }
]
}
```

5.5 User Upload — Validation Rules

When a tenant uploads a CSV or JSON file to create or replace their CoA:

Rule	Validation	Error Message
R1	File must be valid CSV or JSON	"Invalid file format. Upload a CSV or JSON file conforming to the SMS CoA schema."
R2	All 5 root types must be present at Level 1	"Missing root type: {type}. All five root types (Asset, Liability, Equity, Revenue, Expense) are required."
R3	Account codes must be unique within the file	"Duplicate account code: {code} at line {line}."
R4	Level must be 1, 2, 3, or 4	"Invalid level {level} for account {code}. SMS supports levels 1 through 4."
R5	Parent code must exist in the file and be one level above	"Account {code} references parent {parent} which is not found or not at level {expected_level}."
R6	Level 1 accounts must have codes 1–5 matching root types	"Level 1 account code must be 1 (Asset), 2 (Liability), 3 (Equity), 4 (Revenue), or 5 (Expense)."
R7	Level 4 accounts must have is_group = false	"Account {code} is Level 4 and must not be marked as a group."
R8	Levels 1–3 must have is_group = true	"Account {code} is Level {level} and must be marked as a group."
R9	root_type must match the root ancestor's type	"Account {code} has root_type '{type}' but its root ancestor is '{ancestor_type}'."
R10	Maximum 2000 accounts per upload	"Upload contains {count} accounts. Maximum allowed is 2000."
R11	Account code must be alphanumeric (digits recommended)	"Account code '{code}' contains invalid characters. Use digits only."
R12	account_name maximum 200 characters	"Account name exceeds 200 characters at code {code}."
R13	If tenant already has posted journal entries, upload is blocked	"Chart of Accounts cannot be replaced after journal entries have been posted. Add or modify individual accounts instead."

6. C4 — Tenant Chart of Accounts (4-Level Hierarchy)

6.1 Description

The tenant's actual Chart of Accounts — the live account tree that journal entries post against. Created by applying a template, uploading a file, or manual entry. This is the central entity of the financial system.

6.2 Entity Definition — `finance.ChartOfAccounts`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
account_code	nvarchar(20)	NO	—	Account code (e.g., "1111"). Unique per org
account_name	nvarchar(200)	NO	—	Display name
parent_account_id	int	YES	NULL	FK → self. NULL for Level 1 root accounts
materialized_path	nvarchar(500)	NO	—	Full path from root (e.g., "/1/11/111/1111"). Auto-computed on insert/update
level	tinyint	NO	—	1, 2, 3, or 4
root_type	nvarchar(20)	NO	—	'Asset', 'Liability', 'Equity', 'Revenue', 'Expense'
is_group	bit	NO	—	true for Levels 1–3, false for Level 4
account_type	nvarchar(50)	YES	NULL	Sub-classification for Level 4
description	nvarchar(500)	YES	NULL	Help text
external_code	nvarchar(50)	YES	NULL	Account code in an external financial system (for integration)
is_system_account	bit	NO	0	true for system-created accounts that cannot be deleted (e.g., Opening Balance Equity)
is_active	bit	NO	1	Soft-delete. Inactive accounts cannot receive new postings but retain history
has_transactions	bit	NO	0	Cached flag set to true when the first journal entry posts to this account. Prevents deletion
opening_balance_debit	decimal(18,4)	NO	0	Opening balance debit amount (set during opening balance entry)

Column	Type	Nullable	Default	Description
opening_balance_credit	decimal(18,4)	NO	0	Opening balance credit amount
current_balance	decimal(18,4)	NO	0	Running balance (opening + all posted entries). Updated by journal posting engine
sort_order	int	NO	0	Display order within parent
source_template_id	int	YES	NULL	FK → finance.CoaTemplates.id. Which template this account originated from (NULL if manually created or uploaded)
org_id	int	NO	—	FK → core.Organizations. Multi-tenant discriminator
created_at	datetime2(7)	NO	GETUTCDATE()	
created_by	int	YES	NULL	FK → auth.Users
modified_at	datetime2(7)	YES	NULL	
modified_by	int	YES	NULL	FK → auth.Users

Indexes:

Index	Columns	Type	Notes
UX_ChartOfAccounts_Code_Org	account_code, org_id	Unique, Filtered (WHERE is_active = 1)	One account code per tenant
IX_ChartOfAccounts_Parent	parent_account_id	Non-unique	Tree navigation
IX_ChartOfAccounts_OrgId	org_id	Non-unique	Tenant filter (EF Core query filter)
IX_ChartOfAccounts_RootType	org_id, root_type	Non-unique	Filter by account type
IX_ChartOfAccounts_Level	org_id, level	Non-unique	Filter by level
IX_ChartOfAccounts_Path	org_id, materialized_path	Non-unique	Subtree queries with LIKE
IX_ChartOfAccounts_AccountType	org_id, account_type	Non-unique, Filtered (WHERE account_type IS NOT NULL)	Lookup accounts by sub-type
IX_ChartOfAccounts_Active_Leaf	org_id, is_active, is_group	Non-unique, Filtered (WHERE is_active = 1 AND is_group = 0)	Account selectors — postable accounts only

Constraints:

- `level` CHECK: `level BETWEEN 1 AND 4`
- `root_type` CHECK: `root_type IN ('Asset', 'Liability', 'Equity', 'Revenue', 'Expense')`
- `is_group` CHECK: `(level IN (1,2,3) AND is_group = 1) OR (level = 4 AND is_group = 0)`
- `parent_account_id` must be NULL when `level = 1`
- `parent_account_id` must reference an account with `level = current_level - 1` within the same `org_id`
- `opening_balance_debit` and `opening_balance_credit` cannot both be > 0 for the same account
- `materialized_path` must start with `/` and end with the current account's code segment

6.3 Materialized Path Computation

The `materialized_path` is automatically computed by a stored procedure triggered on insert and update:

```sql
-- Example: Account 1111 under parent 111 under 11 under 1
-- materialized_path = '/1/11/111/1111'

-- Subtree query (all accounts under "Current Assets" code 11):
SELECT * FROM finance.ChartOfAccounts
WHERE org_id = @OrgId
  AND materialized_path LIKE '/1/11/%'
ORDER BY materialized_path;

-- Recursive CTE for ancestry:
WITH AccountTree AS (
    SELECT id, account_code, account_name, parent_account_id, level, 0 AS depth
    FROM finance.ChartOfAccounts
    WHERE id = @AccountId AND org_id = @OrgId
    UNION ALL
    SELECT c.id, c.account_code, c.account_name, c.parent_account_id, c.level, at.depth + 1
    FROM finance.ChartOfAccounts c
    INNER JOIN AccountTree at ON c.parent_account_id = at.id
    WHERE c.org_id = @OrgId
)
SELECT * FROM AccountTree ORDER BY depth, sort_order;
```

6.4 System-Created Accounts

When a tenant applies a template or completes initial setup, the system auto-creates one mandatory account if not present in the template:

Account Code	Account Name	Level	Root Type	Account Type	Purpose
3900	Opening Balance Equity	4	Equity	Equity	Temporary balancing account for opening balance entries. Debits and credits from opening balances that do not net to zero are parked here. After initial setup, this account's balance should be zero.

This account has `is_system_account = true` and cannot be deleted or renamed.

6.5 C# Entity Model

```csharp
```

```csharp
namespace SMS.Domain.Entities.Finance;

public class ChartOfAccount : BaseAuditableEntity, IMultiTenant
{
    public int Id { get; set; }
    public string AccountCode { get; set; }          // "1111"
    public string AccountName { get; set; }          // "Petty Cash"
    public int? ParentAccountId { get; set; }        // FK to self
    public string MaterializedPath { get; set; }     // "/1/11/111/1111"
    public byte Level { get; set; }                  // 1, 2, 3, or 4
    public string RootType { get; set; }             // Asset|Liability|Equity|Revenue|Expense
    public bool IsGroup { get; set; }                // true for L1-L3, false for L4
    public string? AccountType { get; set; }         // Cash, Bank, Receivable, etc.
    public string? Description { get; set; }
    public string? ExternalCode { get; set; }        // Code in external system
    public bool IsSystemAccount { get; set; }
    public bool IsActive { get; set; } = true;
    public bool HasTransactions { get; set; }
    public decimal OpeningBalanceDebit { get; set; }
    public decimal OpeningBalanceCredit { get; set; }
    public decimal CurrentBalance { get; set; }
    public int SortOrder { get; set; }
    public int? SourceTemplateId { get; set; }
    public int OrgId { get; set; }

    // Navigation Properties
    public ChartOfAccount? Parent { get; set; }
    public ICollection<ChartOfAccount> Children { get; set; } = new List<ChartOfAccount>();
    public CoaTemplate? SourceTemplate { get; set; }
    public Organization Organization { get; set; }
}
```

6.6 EF Core Configuration

csharp

```
public class ChartOfAccountConfiguration : IEntityTypeConfiguration<ChartOfAccount>
{
    public void Configure(EntityTypeBuilder<ChartOfAccount> builder)
    {
        builder.ToTable("ChartOfAccounts", "finance");
        builder.HasKey(e => e.Id);

        builder.Property(e => e.AccountCode).HasMaxLength(20).IsRequired();
        builder.Property(e => e.AccountName).HasMaxLength(200).IsRequired();
        builder.Property(e => e.MaterializedPath).HasMaxLength(500).IsRequired();
        builder.Property(e => e.RootType).HasMaxLength(20).IsRequired();
        builder.Property(e => e.AccountType).HasMaxLength(50);
        builder.Property(e => e.Description).HasMaxLength(500);
        builder.Property(e => e.ExternalCode).HasMaxLength(50);
        builder.Property(e => e.OpeningBalanceDebit).HasPrecision(18, 4).HasDefaultValue(0);
        builder.Property(e => e.OpeningBalanceCredit).HasPrecision(18, 4).HasDefaultValue(0);
        builder.Property(e => e.CurrentBalance).HasPrecision(18, 4).HasDefaultValue(0);

        builder.HasOne(e => e.Parent)
            .WithMany(e => e.Children)
            .HasForeignKey(e => e.ParentAccountId)
            .OnDelete(DeleteBehavior.Restrict);

        builder.HasOne(e => e.SourceTemplate)
            .WithMany()
            .HasForeignKey(e => e.SourceTemplateId)
            .OnDelete(DeleteBehavior.SetNull);

        builder.HasIndex(e => new { e.AccountCode, e.OrgId })
            .IsUnique()
            .HasFilter("[is_active] = 1");

        builder.HasIndex(e => e.ParentAccountId);
        builder.HasIndex(e => e.OrgId);
        builder.HasIndex(e => new { e.OrgId, e.RootType });
        builder.HasIndex(e => new { e.OrgId, e.Level });
        builder.HasIndex(e => new { e.OrgId, e.MaterializedPath });

        builder.HasQueryFilter(e => e.OrgId == _tenantProvider.OrgId);
    }
}
```

7. C5 — Fiscal Periods

7.1 Description

Fiscal years and their monthly (or custom) periods. Opening balances are entered against a specific fiscal year, and all future journal entries must post within an open fiscal period. Fiscal periods control which date ranges accept postings.

7.2 Entity Definition — `finance.FiscalYears`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
fiscal_year_name	nvarchar(100)	NO	—	Display name (e.g., "FY 2026", "April 2026 – March 2027")
start_date	date	NO	—	First day of the fiscal year
end_date	date	NO	—	Last day of the fiscal year
is_closed	bit	NO	0	true when year-end closing is complete. No further postings allowed
reporting_currency_code	nvarchar(3)	NO	—	FK → lookups.Currencies. Reporting currency for this fiscal year
org_id	int	NO	—	Multi-tenant discriminator
created_at	datetime2(7)	NO	GETUTCDATE()	
created_by	int	YES	NULL	
modified_at	datetime2(7)	YES	NULL	
modified_by	int	YES	NULL	

Indexes:

Index	Columns	Type
UX_FiscalYears_Org_Name	org_id, fiscal_year_name	Unique
IX_FiscalYears_Org_Dates	org_id, start_date, end_date	Non-unique

Constraints:

- `end_date` must be greater than `start_date`
- Fiscal year date ranges must not overlap within the same `org_id`
- `reporting_currency_code` must exist in `lookups.Currencies`

7.3 Entity Definition — `finance.FiscalPeriods`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
fiscal_year_id	int	NO	—	FK → finance.FiscalYears.id
period_name	nvarchar(100)	NO	—	Display name (e.g., "January 2026", "Period 1")
period_number	tinyint	NO	—	Sequential number within the year (1–13). Period 13 is reserved for year-end adjustments
start_date	date	NO	—	First day of the period
end_date	date	NO	—	Last day of the period
status	nvarchar(20)	NO	`'Open'`	'Open', 'Closed', 'Locked'. Only 'Open' periods accept postings. 'Locked' is a soft-close (reversible). 'Closed' is final
org_id	int	NO	—	Multi-tenant discriminator
created_at	datetime2(7)	NO	GETUTCDATE()	
modified_at	datetime2(7)	YES	NULL	

Indexes:

Index	Columns	Type
UX_FiscalPeriods_Year_Number	fiscal_year_id, period_number, org_id	Unique
IX_FiscalPeriods_Org_Status	org_id, status	Non-unique
IX_FiscalPeriods_Org_Dates	org_id, start_date, end_date	Non-unique

Constraints:

- `period_number` CHECK: `period_number BETWEEN 1 AND 13`
- `status` CHECK: `status IN ('Open', 'Closed', 'Locked')`
- Period date ranges must fall within their parent fiscal year's date range
- Period date ranges must not overlap within the same fiscal year

7.4 Auto-Generation of Periods

When a tenant creates a fiscal year, the system auto-generates 12 monthly periods:

```
Fiscal Year: April 2026 – March 2027
  Period 1:  April 2026      (2026-04-01 to 2026-04-30)
  Period 2:  May 2026        (2026-05-01 to 2026-05-31)
  Period 3:  June 2026       (2026-06-01 to 2026-06-30)
  ...
  Period 12: March 2027      (2027-03-01 to 2027-03-31)
  Period 13: Year-End Adj.   (2027-03-31 to 2027-03-31)  ← adjustment period
```

The tenant can customize period boundaries (e.g., 4-4-5 calendar, 13 periods) but must cover the entire fiscal year without gaps or overlaps.

---

8. C6 — Accounting Defaults

8.1 Description

Default accounts that the system uses when auto-generating journal entries from business events. When a sales invoice is posted, the system needs to know which Accounts Receivable, Revenue, and Tax accounts to debit/credit. These defaults are configured per tenant after CoA setup.

8.2 Entity Definition — `finance.AccountingDefaults`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
default_key	nvarchar(50)	NO	—	Unique identifier for this default (see list below)
default_label	nvarchar(200)	NO	—	Human-readable label (e.g., "Default Accounts Receivable")
account_id	int	YES	NULL	FK → finance.ChartOfAccounts.id. The assigned default account. NULL = not yet configured
is_required	bit	NO	1	If true, this default must be set before CoA setup is marked complete
category	nvarchar(50)	NO	—	Grouping for UI display: 'Receivables', 'Payables', 'Banking', 'Tax', 'Revenue', 'CostOfSales', 'Inventory', 'Equity'
help_text	nvarchar(500)	YES	NULL	Explanation shown in the UI
org_id	int	NO	—	Multi-tenant discriminator
created_at	datetime2(7)	NO	GETUTCDATE()	
modified_at	datetime2(7)	YES	NULL	
modified_by	int	YES	NULL	

Indexes:

Index	Columns	Type
UX_AccountingDefaults_Key_Org	default_key, org_id	Unique
IX_AccountingDefaults_AccountId	account_id	Non-unique
IX_AccountingDefaults_OrgId	org_id	Non-unique

Constraints:

- `account_id` must reference a Level 4 (leaf) account in the same `org_id`

- `account_id` must reference an active account (`is_active = 1`)
- `default_key` CHECK: must be one of the recognized keys (see below)

8.3 Default Account Keys

default_key	default_label	Category	Required	Expected Account Type	Description
DEFAULT_RECEIVABLE	Accounts Receivable	Receivables	Yes	Receivable	Customer balances for unpaid invoices
DEFAULT_PAYABLE	Accounts Payable	Payables	Yes	Payable	Supplier balances for unpaid bills
DEFAULT_CASH	Cash Account	Banking	Yes	Cash	Primary cash account
DEFAULT_BANK	Bank Account	Banking	Yes	Bank	Primary bank account
DEFAULT_REVENUE	Sales Revenue	Revenue	Yes	Revenue	Default revenue account for sales
DEFAULT_COGS	Cost of Goods Sold	CostOfSales	Yes	CostOfGoodsSold	Default COGS account
DEFAULT_TAX_RECEIVABLE	Tax Receivable (Input VAT)	Tax	No	Tax	Input tax / VAT paid on purchases
DEFAULT_TAX_PAYABLE	Tax Payable (Output VAT)	Tax	No	Tax	Output tax / VAT collected on sales
DEFAULT_INVENTORY	Inventory	Inventory	No	OtherCurrentAsset	Default inventory holding account

default_key	default_label	Category	Required	Expected Account Type	Description
DEFAULT_PURCHASE	Purchase Account	CostOfSales	No	Expense	Default account for purchase transactions
DEFAULT_DISCOUNT_GIVEN	Discount Given	Revenue	No	Revenue	Discounts given to customers
DEFAULT_DISCOUNT_RECEIVED	Discount Received	CostOfSales	No	OtherIncome	Discounts received from suppliers
DEFAULT_RETAINED_EARNINGS	Retained Earnings	Equity	Yes	Equity	Year-end profit/loss closing account
DEFAULT_OPENING_BALANCE	Opening Balance Equity	Equity	Yes	Equity	Auto-assigned to account 3900
DEFAULT_EXCHANGE_GAIN_LOSS	Exchange Gain/Loss	Revenue	No	OtherIncome	Realized foreign exchange differences
DEFAULT_ROUNDING	Rounding Adjustment	Revenue	No	OtherExpense	Rounding differences in payments

When a template is applied, accounts marked with `is_default_account = true` in the template items automatically populate the corresponding `AccountingDefaults` rows.

9. C7 — CoA Setup Status & Module Gate

9.1 Description

Tracks whether a tenant has completed the Chart of Accounts setup process. Financial modules check this status before allowing any operation. This is the gatekeeper that ensures no financial transaction occurs without a properly configured CoA.

9.2 Entity Definition — `finance.CoaSetupStatuses`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
status	nvarchar(30)	NO	`'NotStarted'`	Current step in the setup process
template_applied_at	datetime2(7)	YES	NULL	When a template was applied
template_id	int	YES	NULL	FK → finance.CoaTemplates.id. Which template was applied (NULL if manual/upload)
accounts_created_count	int	NO	0	Number of accounts in the tenant's CoA
fiscal_year_configured	bit	NO	0	Has at least one fiscal year been created?
defaults_configured	bit	NO	0	Have all required accounting defaults been assigned?
opening_balances_entered	bit	NO	0	Have opening balances been entered (even if all zero)?
opening_balances_balanced	bit	NO	0	Do total opening debits = total opening credits?
setup_completed_at	datetime2(7)	YES	NULL	When setup was marked complete
setup_completed_by	int	YES	NULL	FK → auth.Users
conversion_date	date	YES	NULL	The date chosen as the starting point. Opening balances represent the position as of this date
org_id	int	NO	—	Multi-tenant discriminator. One row per tenant
created_at	datetime2(7)	NO	GETUTCDATE()	
modified_at	datetime2(7)	YES	NULL	

Indexes:

Index	Columns	Type
UX_CoaSetupStatuses_OrgId	org_id	Unique — one status row per tenant

9.3 Setup Status State Machine

Status Values:

Status	Description	Can Move To
NotStarted	Tenant has activated Finance module but not begun setup	AccountsCreated
AccountsCreated	Template applied, file uploaded, or accounts manually added	FiscalYearSet, AccountsCreated (re-import/re-apply allowed before fiscal year)
FiscalYearSet	At least one fiscal year with periods has been created	DefaultsSet
DefaultsSet	All required accounting defaults assigned to valid Level 4 accounts	BalancesEntered
BalancesEntered	Opening balances have been entered for at least one account	Complete
Complete	Setup finalized. Financial modules are unlocked. Conversion date is locked	— (terminal state)

Transition Rules:

- Steps can be revisited before reaching `Complete` (e.g., re-apply a different template resets status to `AccountsCreated`)
- Once status reaches `Complete`, the conversion date and the 5 Level 1 root accounts are locked — they cannot be deleted or re-typed
- The `Complete` transition requires `opening_balances_balanced = true` (debits = credits) OR the tenant explicitly acknowledges the imbalance by checking "My opening balances include an intentional difference to Opening Balance Equity"

9.4 Module Gate Middleware

Every API endpoint under `/api/finance/*` (except CoA Setup endpoints) passes through the `FinanceSetupGateMiddleware`:

```csharp
```

```
public class FinanceSetupGateMiddleware
{
    public async Task InvokeAsync(HttpContext context)
    {
        if (context.Request.Path.StartsWithSegments("/api/finance") &&
            !IsSetupEndpoint(context.Request.Path))
        {
            var setupStatus = await _setupService
                .GetSetupStatusAsync(context.GetTenantOrgId());

            if (setupStatus.Status != "Complete")
            {
                context.Response.StatusCode = 403;
                await context.Response.WriteAsJsonAsync(new
                {
                    error = "FINANCE_SETUP_REQUIRED",
                    message = "Chart of Accounts setup must be completed before using financi
                    current_status = setupStatus.Status,
                    redirect_url = "/finance/setup"
                });
                return;
            }
        }

        await _next(context);
    }
}
```

On the UI side, when a user navigates to any financial module page and setup is incomplete, the React router redirects to `/finance/setup` with a banner explaining which step is needed.

---

10. C8 — Opening Balances

10.1 Description

Opening balances represent the tenant's financial position at the conversion date — the day before they start using SMS for financial transactions. The system captures debit or credit balances against each Level 4 account and records them as a special journal entry.

10.2 Entity Definition — `finance.OpeningBalanceEntries`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
account_id	int	NO	—	FK → finance.ChartOfAccounts.id. Must be a Level 4 (leaf) account
debit_amount	decimal(18,4)	NO	0	Opening debit balance
credit_amount	decimal(18,4)	NO	0	Opening credit balance
notes	nvarchar(500)	YES	NULL	Optional note explaining the balance
conversion_date	date	NO	—	The conversion date (copied from CoaSetupStatuses for data integrity)
entry_status	nvarchar(20)	NO	`'Draft'`	'Draft' or 'Posted'. Draft balances can be modified. Posted balances are locked
posted_at	datetime2(7)	YES	NULL	When the balance was finalized
posted_by	int	YES	NULL	FK → auth.Users
org_id	int	NO	—	Multi-tenant discriminator
created_at	datetime2(7)	NO	GETUTCDATE()	
created_by	int	YES	NULL	
modified_at	datetime2(7)	YES	NULL	
modified_by	int	YES	NULL	

Indexes:

Index	Columns	Type
UX_OpeningBalanceEntries_Account_Org	account_id, org_id	Unique — one opening balance entry per account per tenant
IX_OpeningBalanceEntries_OrgId	org_id	Non-unique
IX_OpeningBalanceEntries_Status	org_id, entry_status	Non-unique

Constraints:

- `account_id` must reference a Level 4 account (`is_group = false`) within the same `org_id`
- `debit_amount` and `credit_amount` cannot both be > 0 for the same row
- `debit_amount` and `credit_amount` must be >= 0
- `entry_status` CHECK: `entry_status IN ('Draft', 'Posted')`
- Posted entries cannot be modified (enforced by application logic)

10.3 Opening Balance Posting Logic

When the tenant finalizes opening balances (transition from Draft to Posted):

1. **Validate Balance** — Sum all debit amounts and all credit amounts across all entries for the org. If total debits ≠ total credits, the difference is automatically posted to the Opening Balance Equity account (3900). The system displays a warning: "Opening balances have a difference of {amount}. This has been posted to Opening Balance Equity (3900)."
2. **Create Journal Entry** — The system creates a journal entry with type = `OPENING_BALANCE`:

```
Journal Entry: OPENING_BALANCE
Date: [Conversion Date]
Reference: "Opening Balance Entry — {Conversion Date}"
Status: Posted

Lines:
  Debit  Petty Cash (1111)                5,000.00
  Debit  Cash at Bank (1113)            150,000.00
  Debit  Accounts Receivable (1121)      45,000.00
  Debit  Raw Materials (1131)            80,000.00
  Credit Accounts Payable (2111)                      65,000.00
  Credit Bank Loan (2211)                            100,000.00
  Credit Owner's Capital (3100)                      110,000.00
  Credit Opening Balance Equity (3900)                 5,000.00
         ──────────────────              ──────────────
  Total: 280,000.00                      280,000.00    ✓ Balanced
```

3. **Update ChartOfAccounts** — Set `opening_balance_debit` and `opening_balance_credit` on each affected account. Set `current_balance` = debit − credit (for debit-normal accounts) or credit − debit (for credit-normal accounts). Set `has_transactions = true`.
4. **Update CoaSetupStatuses** — Set `opening_balances_entered = true`, `opening_balances_balanced = true` (if balanced or difference accepted).

10.4 Conversion Date Rules

- The conversion date is the **last day before the tenant starts using SMS** for financial transactions
- Example: If the tenant starts using SMS on 2026-10-01, the conversion date is 2026-09-30
- The conversion date must fall within the first fiscal year's date range
- After opening balances are posted, the conversion date cannot be changed
- All opening balance entries share the same conversion date (one per tenant)

---

11. C9 — CoA Setup Wizard & Guide

11.1 Description

A multi-step guided wizard that walks the tenant through the complete Chart of Accounts setup process. Includes contextual help, progress tracking, and an optional tutorial guide. The wizard is presented when a user accesses any finance module page and the CoA setup is not complete.

11.2 Wizard Steps

Step	Title	Description	Status Source
1	Choose Template	Select a CoA template by country and business type, or choose to upload a custom file	CoaSetupStatuses.status ⩾ AccountsCreated
2	Review & Customize Accounts	View the account tree, add/edit/delete accounts at Levels 2–4	CoaSetupStatuses.accounts_created_count > 0
3	Set Up Fiscal Year	Define the first fiscal year start/end dates, review auto-generated periods	CoaSetupStatuses.fiscal_year_configured = true
4	Configure Default Accounts	Assign accounts to each required default (receivable, payable, cash, bank, revenue, COGS, retained earnings)	CoaSetupStatuses.defaults_configured = true
5	Enter Opening Balances	Enter debit or credit balances for each Level 4 account. Can skip for new companies with zero balances	CoaSetupStatuses.opening_balances_entered = true
6	Review & Complete	Summary of setup, balance verification, finalize button	CoaSetupStatuses.status = Complete

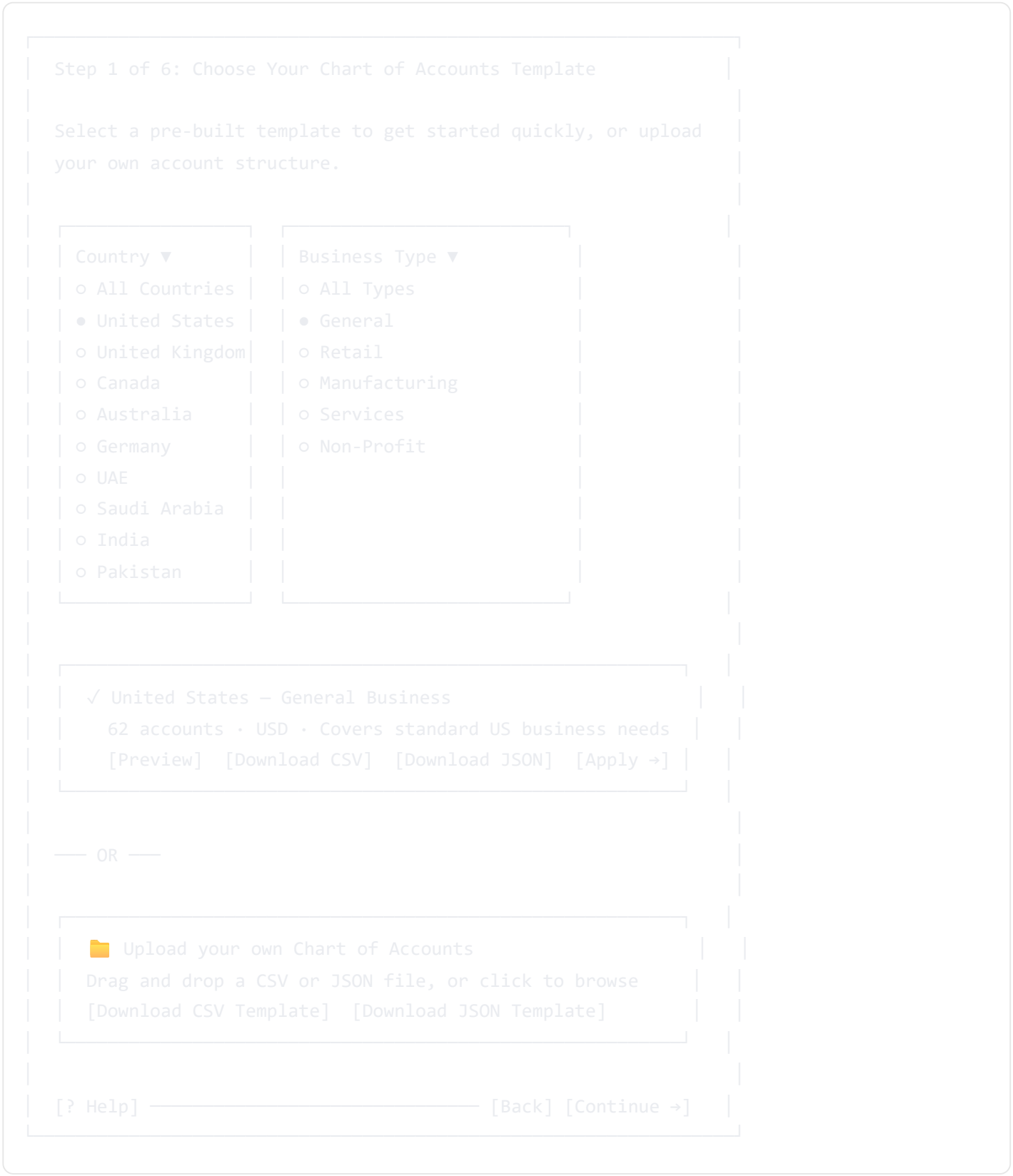
11.3 Contextual Help System

Each wizard step includes an expandable help panel with:

- **What is this?** — Plain-language explanation of the concept (e.g., "A Chart of Accounts is the organized list of all accounts your company uses to track money coming in, going out, and where it sits")
- **Why does it matter?** — Business relevance (e.g., "Default accounts tell the system where to automatically record common transactions like sales and purchases. Without them, you would have to manually select accounts every time")
- **Tips** — Practical guidance (e.g., "Most businesses start with 40–80 accounts. You can always add more later, but removing accounts that have transactions is not possible")
- **Example** — A concrete illustration using a relatable business scenario

11.4 Template Selection UI

Step 1 presents a two-level filter:



11.5 Opening Balance Entry UI

Step 5 presents a flat table of all Level 4 accounts grouped by root type, with debit/credit input fields:

Step 5 of 6: Enter Opening Balances

Conversion Date: [2026-09-30]
Enter the balance of each account as of this date.

Code	Account Name	Debit	Credit
	▼ ASSETS		
1111	Petty Cash	5,000.00	
1112	Cash in Hand	12,000.00	
1113	Cash at Bank — Operating	150,000.00	
1121	Accounts Receivable	45,000.00	
1131	Raw Materials	80,000.00	
	▼ LIABILITIES		
2111	Accounts Payable		65,000.00
2211	Bank Loan		100,000.00
	▼ EQUITY		
3100	Owner's Capital		110,000.00
	▼ REVENUE		
	(no opening balance for P&L accounts in new fiscal year)		
	▼ EXPENSES		
	(no opening balance for P&L accounts in new fiscal year)		
	TOTAL	292,000.00	275,000.00
	DIFFERENCE	17,000.00	
	→ To Opening Balance Eq.		17,000.00

⚠ Debits and credits differ by 17,000.00. The difference will be posted to Opening Balance Equity (3900).

☐ I confirm this difference is intentional
☐ Skip opening balances (new company with zero balances)

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│  [? Help] ──────────────────── [Save Draft] [Back] [Post & Continue]│
└──────────────────────────────────────────────────────────────────────┘
```

---

12. C10 — External Financial System Integration Hooks

12.1 Description

SMS tenants may run their primary accounting in an external system (QuickBooks, Xero, ERPNext, Tally, Sage, etc.) and use SMS for supply chain and sales operations. This change establishes the data structure for mapping SMS Chart of Accounts entries to their counterparts in external systems, enabling future bidirectional sync of chart data, journal entries, and balances.

This addendum defines the **mapping table and configuration only** — the actual sync connectors (OAuth authentication, API calls, data transformation) are deferred to a dedicated integration addendum.

12.2 Entity Definition — `finance.ExternalSystemMappings`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
system_type	nvarchar(50)	NO	—	External system identifier (see list below)
system_name	nvarchar(200)	NO	—	Display name for this connection (e.g., "Our QuickBooks Online", "Xero — UK Entity")
account_id	int	NO	—	FK → finance.ChartOfAccounts.id. The SMS account
external_account_code	nvarchar(100)	NO	—	Account code/ID in the external system
external_account_name	nvarchar(200)	YES	NULL	Account name in the external system (for display)
sync_direction	nvarchar(20)	NO	`'None'`	'None', 'Inbound', 'Outbound', 'Bidirectional'
is_active	bit	NO	1	Enable/disable this mapping
last_synced_at	datetime2(7)	YES	NULL	Timestamp of last successful sync
sync_status	nvarchar(30)	YES	NULL	'Success', 'Error', 'Pending', NULL (never synced)
sync_error_message	nvarchar(1000)	YES	NULL	Last sync error detail
connection_config_json	nvarchar(max)	YES	NULL	Encrypted JSON blob for connection-specific metadata (tenant ID, company file ref, etc.). Sensitive credentials stored in system vault, not here
org_id	int	NO	—	Multi-tenant discriminator
created_at	datetime2(7)	NO	GETUTCDATE()	

Column	Type	Nullable	Default	Description
created_by	int	YES	NULL	
modified_at	datetime2(7)	YES	NULL	
modified_by	int	YES	NULL	

Indexes:

Index	Columns	Type
UX_ExtSysMappings_Sys_Account_Org	system_type, account_id, org_id	Unique — one mapping per system per SMS account per tenant
IX_ExtSysMappings_AccountId	account_id	Non-unique
IX_ExtSysMappings_OrgId	org_id	Non-unique
IX_ExtSysMappings_SyncDirection	org_id, system_type, sync_direction	Non-unique, Filtered (WHERE is_active = 1)

Constraints:

- `system_type` CHECK: must be one of the recognized values (see below)
- `sync_direction` CHECK: `sync_direction IN ('None', 'Inbound', 'Outbound', 'Bidirectional')`
- `account_id` must reference an active, Level 4 account within the same `org_id`

12.3 Supported External System Types

system_type	Display Name	Notes
QUICKBOOKS_ONLINE	QuickBooks Online	Intuit's cloud accounting. REST API v3
QUICKBOOKS_DESKTOP	QuickBooks Desktop	On-premise. Uses Web Connector / SDK
XERO	Xero	Cloud accounting. OAuth 2.0 REST API
ERPNEXT	ERPNext	Open-source ERP. REST API
TALLY	Tally Prime	Widely used in South Asia. XML-based sync
SAGE	Sage (Intacct / 50 / 200)	Multiple Sage products share this type
SAP_B1	SAP Business One	SAP's SME ERP. Service Layer / DI API
DYNAMICS_BC	Microsoft Dynamics 365 Business Central	Cloud ERP. OData REST API
ZOHO_BOOKS	Zoho Books	Cloud accounting. REST API
WAVE	Wave Accounting	Free cloud accounting (Intuit-owned). REST API
ODOO	Odoo Accounting	Open-source ERP. XML-RPC / JSON-RPC
CUSTOM	Custom / Other	Generic type for unlisted systems

12.4 Integration Architecture (Conceptual — Connector Implementation Deferred)

12.5 External Code on ChartOfAccounts

The `external_code` field on `finance.ChartOfAccounts` provides a quick-access mapping without joining the mappings table. This is useful for single-system tenants. The `ExternalSystemMappings`

table supports multi-system tenants who map the same SMS account to different codes in different external systems.

---

13. Database Migrations

13.1 Migration Summary

Migration ID	Name	Description	Change Item
M1	AddFinanceSchema	Create `finance` schema, update Module Catalog	C1
M2	CreateCoaTemplates	Create `finance.CoaTemplates` table	C2
M3	SeedCoaTemplates	Seed 16 system templates into CoaTemplates	C2
M4	CreateCoaTemplateItems	Create `finance.CoaTemplateItems` table	C3
M5	SeedGlobalGeneralTemplate	Seed account items for GLOBAL_GENERAL template (~65 accounts)	C3
M6	SeedUSGeneralTemplate	Seed account items for US_GENERAL template (~70 accounts)	C3
M7	CreateChartOfAccounts	Create `finance.ChartOfAccounts` table with all indexes	C4
M8	CreateFiscalYearsAndPeriods	Create `finance.FiscalYears` and `finance.FiscalPeriods` tables	C5
M9	CreateAccountingDefaults	Create `finance.AccountingDefaults` table, seed 16 default key rows per new tenant	C6
M10	CreateCoaSetupStatuses	Create `finance.CoaSetupStatuses` table	C7
M11	CreateOpeningBalanceEntries	Create `finance.OpeningBalanceEntries` table	C8
M12	CreateExternalSystemMappings	Create `finance.ExternalSystemMappings` table	C10

13.2 Migration Execution Order

Migrations run sequentially (M1 → M12). Dependencies:

- M4 depends on M2 (CoaTemplateItems.template_id → CoaTemplates.id)
- M5, M6 depend on M2 and M4 (seed data references template and item tables)
- M7 depends on M2 (ChartOfAccounts.source_template_id → CoaTemplates.id)
- M9 depends on M7 (AccountingDefaults.account_id → ChartOfAccounts.id)
- M10 depends on M2 (CoaSetupStatuses.template_id → CoaTemplates.id)
- M11 depends on M7 (OpeningBalanceEntries.account_id → ChartOfAccounts.id)
- M12 depends on M7 (ExternalSystemMappings.account_id → ChartOfAccounts.id)

13.3 Seed Data Notes

- Template item seed data (M5, M6) is authored as embedded JSON resources in the migration assembly, deserialized into `CoaTemplateItems` rows. Additional country templates (UK, Canada, India, Pakistan, UAE, etc.) are seeded in follow-up migrations — one migration per template to keep each migration small and reviewable.
- `AccountingDefaults` rows are seeded per tenant when the Finance module is activated, not globally. The seed creates the 16 default key rows with `account_id = NULL` (unconfigured), ready for the tenant to assign accounts during setup.

---

14. API Endpoints

14.1 CoA Template Endpoints

Method	Route	Description	Auth
GET	`/api/finance/templates`	List available templates with country/business-type filters	Authenticated
GET	`/api/finance/templates/{id}`	Get template detail with account tree	Authenticated
GET	`/api/finance/templates/{id}/download?format=csv`	Download template as CSV file	Authenticated
GET	`/api/finance/templates/{id}/download?format=json`	Download template as JSON file	Authenticated
GET	`/api/finance/templates/{id}/preview`	Get flattened account list for preview (no download trigger)	Authenticated

14.2 Chart of Accounts Endpoints

Method	Route	Description	Auth
GET	`/api/finance/chart-of-accounts`	Get tenant's account tree (hierarchical)	Authenticated
GET	`/api/finance/chart-of-accounts/flat`	Get tenant's accounts as flat list with filters (level, root_type, is_group, is_active)	Authenticated
GET	`/api/finance/chart-of-accounts/{id}`	Get single account detail	Authenticated
POST	`/api/finance/chart-of-accounts`	Create a new account (validates hierarchy, level, parent)	Authenticated + Finance.Write
PUT	`/api/finance/chart-of-accounts/{id}`	Update account (name, description, external_code, sort_order). Code and root_type immutable after creation	Authenticated + Finance.Write
DELETE	`/api/finance/chart-of-accounts/{id}`	Soft-delete account (only if no children, no transactions, not system account)	Authenticated + Finance.Write
POST	`/api/finance/chart-of-accounts/apply-template`	Apply a template to create the tenant's CoA. Body: `{ template_id: int }`	Authenticated + Finance.Admin
POST	`/api/finance/chart-of-accounts/upload`	Upload CSV or JSON file to create/replace CoA. Multipart form data	Authenticated + Finance.Admin
GET	`/api/finance/chart-of-accounts/export?format=csv`	Export tenant's CoA as CSV	Authenticated
GET	`/api/finance/chart-of-accounts/export?format=json`	Export tenant's CoA as JSON	Authenticated

14.3 Fiscal Year & Period Endpoints

Method	Route	Description	Auth
GET	`/api/finance/fiscal-years`	List fiscal years for the tenant	Authenticated
POST	`/api/finance/fiscal-years`	Create fiscal year (auto-generates 12 monthly periods + 1 adjustment)	Authenticated + Finance.Admin
PUT	`/api/finance/fiscal-years/{id}`	Update fiscal year name, currency. Cannot change dates if periods have postings	Authenticated + Finance.Admin
GET	`/api/finance/fiscal-years/{id}/periods`	List periods within a fiscal year	Authenticated
PUT	`/api/finance/fiscal-periods/{id}`	Update period status (Open/Locked/Closed), name, dates	Authenticated + Finance.Admin

14.4 Accounting Defaults Endpoints

Method	Route	Description	Auth
GET	`/api/finance/accounting-defaults`	List all defaults with assigned accounts	Authenticated
PUT	`/api/finance/accounting-defaults/{default_key}`	Assign an account to a default. Body: `{ account_id: int }`	Authenticated + Finance.Admin
POST	`/api/finance/accounting-defaults/validate`	Check if all required defaults are assigned and valid	Authenticated

14.5 Opening Balance Endpoints

Method	Route	Description	Auth
GET	`/api/finance/opening-balances`	List all opening balance entries for the tenant	Authenticated
PUT	`/api/finance/opening-balances/batch`	Batch update opening balance entries. Body: array of `{ account_id, debit_amount, credit_amount, notes }`	Authenticated + Finance.Admin
POST	`/api/finance/opening-balances/post`	Post (finalize) all draft opening balances. Creates the OPENING_BALANCE journal entry	Authenticated + Finance.Admin
GET	`/api/finance/opening-balances/summary`	Get totals: total debits, total credits, difference, balanced status	Authenticated

14.6 Setup Status Endpoints

Method	Route	Description	Auth
GET	`/api/finance/setup/status`	Get current setup status for the tenant	Authenticated
POST	`/api/finance/setup/complete`	Mark setup as complete (validates all prerequisites)	Authenticated + Finance.Admin
POST	`/api/finance/setup/reset`	Reset setup to NotStarted (only if no journal entries exist beyond opening balances)	Authenticated + Finance.Admin

14.7 External System Mapping Endpoints

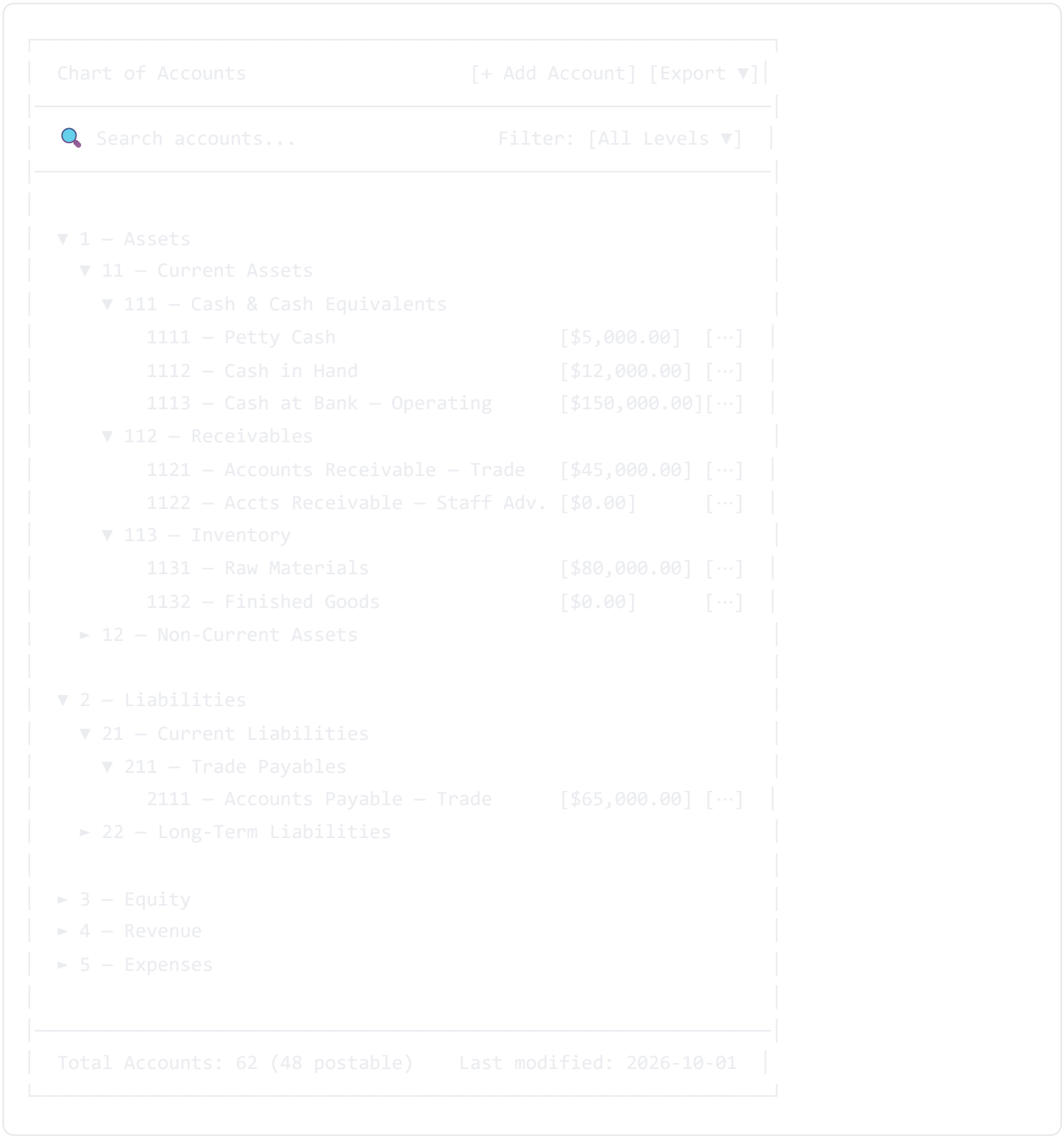
Method	Route	Description	Auth
GET	`/api/finance/external-mappings`	List all mappings for the tenant, optionally filtered by system_type	Authenticated
POST	`/api/finance/external-mappings`	Create a new mapping	Authenticated + Finance.Admin
PUT	`/api/finance/external-mappings/{id}`	Update mapping (external code, direction, active flag)	Authenticated + Finance.Admin
DELETE	`/api/finance/external-mappings/{id}`	Delete a mapping	Authenticated + Finance.Admin
POST	`/api/finance/external-mappings/import`	Import mappings from a CSV file (columns: sms_account_code, external_code, external_name, direction)	Authenticated + Finance.Admin

---

15. UI Wireframes & Specifications

15.1 Finance Module Dashboard (Post-Setup)

15.2 Chart of Accounts — Tree View



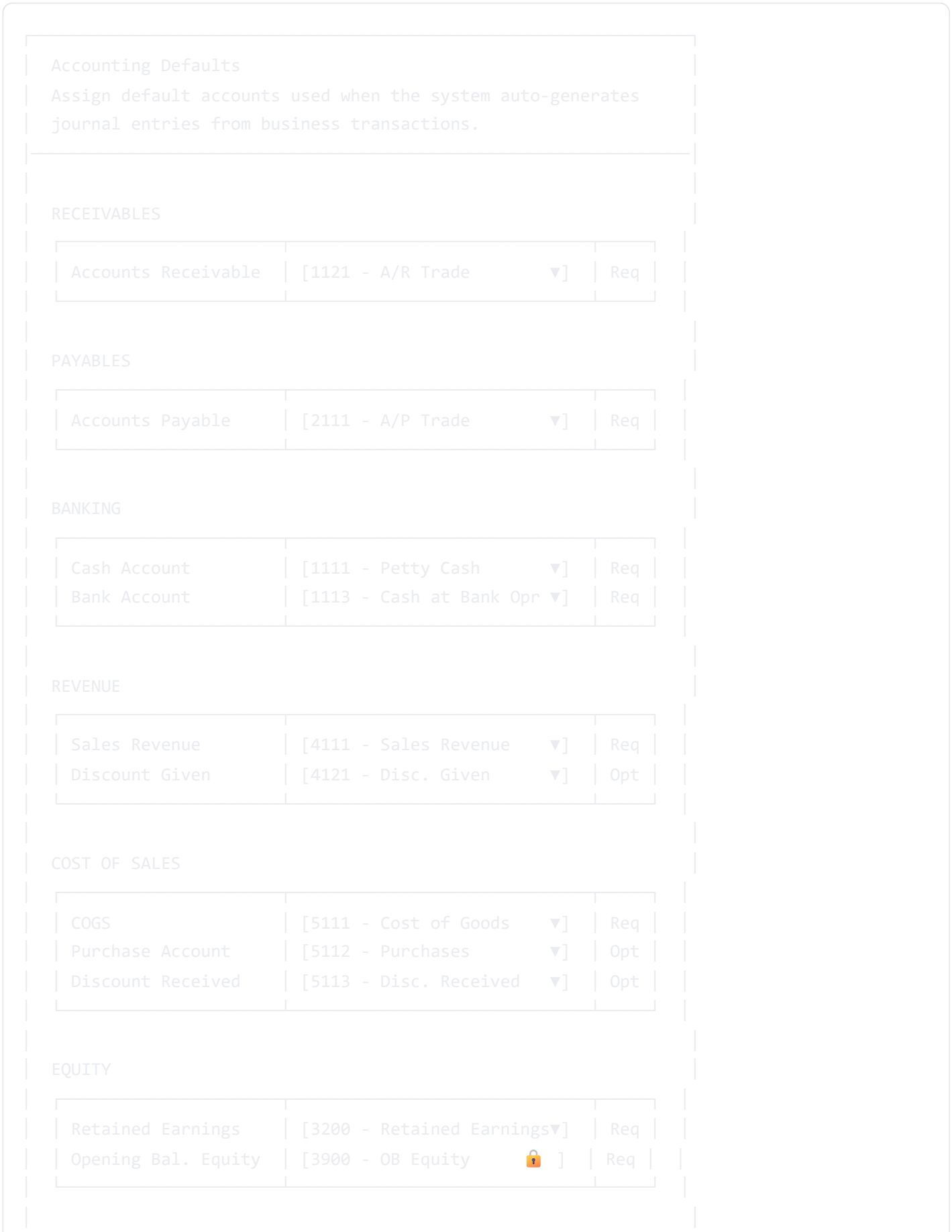
15.3 Add / Edit Account Dialog

Validation Notes:

- Account code auto-suggests next available code under the selected parent
- Root type is auto-inherited from the parent account and read-only
- Level is auto-computed from the parent's level + 1 and read-only
- `is_group` is auto-set: true for Levels 1–3, false for Level 4
- Account type dropdown shows only types valid for the root_type (e.g., under Assets: Cash, Bank, Receivable, FixedAsset, OtherCurrentAsset, AccumulatedDepreciation)

15.4 Fiscal Year Setup Panel

15.5 Accounting Defaults Configuration Panel



```
[Validate All]                                    [Save Defaults]

✓ 7 of 7 required defaults assigned
✓ All assignments reference active Level 4 accounts
```

15.6 External System Mappings Panel

```
External System Integrations                          [+ Add Mapping]
─────────────────────────────────────────────────────────────────────
System: [QuickBooks Online ▼]  Connection: "Our QBO — US"

┌──────┬──────────────────────┬──────────┬──────────────┬──────────┐
│ SMS  │ SMS Account          │ Ext Code │ Ext Name     │ Direct   │
├──────┼──────────────────────┼──────────┼──────────────┼──────────┤
│ 1111 │ Petty Cash           │ 1000     │ Petty Cash   │ Out  ▼   │
│ 1113 │ Cash at Bank         │ 1020     │ Checking Acc │ Bidir▼   │
│ 1121 │ A/R — Trade          │ 1200     │ Accts Recv   │ Bidir▼   │
│ 2111 │ A/P — Trade          │ 2000     │ Accts Pay    │ Bidir▼   │
│ 4111 │ Sales Revenue        │ 4000     │ Sales        │ Out  ▼   │
└──────┴──────────────────────┴──────────┴──────────────┴──────────┘

[Import from CSV]  [Export Mappings]

Last Sync: Never                        Status: Not Connected
(Connectors are configured in a future addendum)
```

---

16. Business Rules

16.1 Chart of Accounts Rules

Rule ID	Rule	Enforcement
BR-COA-001	SMS enforces exactly 4 levels in the Chart of Accounts. Level 1 is fixed (5 root types). Levels 2–4 are user-configurable	CHECK constraint on `level BETWEEN 1 AND 4`, application validation
BR-COA-002	Only Level 4 accounts (is_group = false) can receive journal postings	Application-level check in journal entry creation service
BR-COA-003	An account's root_type is inherited from its root ancestor and cannot be changed	Computed from Level 1 parent on insert, immutable after
BR-COA-004	Account codes must be unique per tenant (org_id) among active accounts	Unique filtered index on (account_code, org_id) WHERE is_active = 1
BR-COA-005	An account cannot be deleted if it has children	Application-level check: `Children.Any()` → block
BR-COA-006	An account cannot be deleted if has_transactions = true	Application-level check → block
BR-COA-007	System accounts (is_system_account = true) cannot be deleted or have root_type/code changed	Application-level check → block
BR-COA-008	When a template is applied, Level 1 accounts (5 root types) are always created regardless of template content	Application logic auto-creates root types
BR-COA-009	An account's parent must be exactly one level above (parent.level = account.level - 1)	Application validation on create/update
BR-COA-010	An account's parent must have the same root_type	Application validation on create/update

Rule ID	Rule	Enforcement
BR-COA-011	materialized_path is auto-computed on insert and update and must not be set directly by the user	Application logic, not exposed in API request model
BR-COA-012	A tenant can have at most 2000 accounts (configurable system setting)	Application-level count check on create/upload

16.2 Template Rules

Rule ID	Rule	Enforcement
BR-TPL-001	System templates (org_id = NULL) are read-only for tenants	Application-level authorization check
BR-TPL-002	Templates must contain all 5 root types at Level 1	Validation on template creation/seed
BR-TPL-003	Template items must form a valid tree (no orphans, no cycles)	Validation on seed/upload
BR-TPL-004	A template can be applied only when CoA setup status is NotStarted or AccountsCreated (before fiscal year is set)	Application-level state check
BR-TPL-005	Applying a template replaces all existing accounts for the tenant (if any)	Application logic: DELETE existing + INSERT from template

16.3 Fiscal Period Rules

Rule ID	Rule	Enforcement
BR-FP-001	Fiscal year date ranges must not overlap within the same tenant	Application validation on create
BR-FP-002	Periods must cover the entire fiscal year without gaps or overlaps	Application validation on period create/update
BR-FP-003	Only Open periods accept journal postings	Application-level check in journal entry service
BR-FP-004	A period cannot be Closed if it contains unposted draft entries	Application-level check on status change
BR-FP-005	A Closed period cannot be reopened. A Locked period can be reopened to Open	Application-level state transition check
BR-FP-006	A fiscal year cannot be Closed until all its periods are Closed	Application-level check

16.4 Opening Balance Rules

Rule ID	Rule	Enforcement
BR-OB-001	Opening balances can only be entered for Level 4 (leaf) accounts	FK constraint + application validation
BR-OB-002	Each account can have at most one opening balance entry per tenant	Unique index on (account_id, org_id)
BR-OB-003	An opening balance entry cannot have both debit > 0 and credit > 0	Application validation
BR-OB-004	Draft opening balances can be modified. Posted opening balances are immutable	Application-level status check on update
BR-OB-005	When total debits ≠ total credits, the difference is posted to Opening Balance Equity (3900)	Application logic during post operation
BR-OB-006	The conversion date must fall within the first fiscal year's date range	Application validation
BR-OB-007	Revenue and Expense accounts typically have zero opening balance in a new fiscal year. The system warns but does not prevent entry	UI warning, no hard block

16.5 Module Gate Rules

Rule ID	Rule	Enforcement
BR-MG-001	All finance API endpoints (except setup endpoints) require CoaSetupStatuses.status = 'Complete'	FinanceSetupGateMiddleware
BR-MG-002	Finance UI pages (except setup wizard) redirect to /finance/setup when status ≠ Complete	React route guard
BR-MG-003	Setup completion requires: accounts_created_count > 0, fiscal_year_configured = true, defaults_configured = true, opening_balances_entered = true	Application validation in complete endpoint
BR-MG-004	Once Complete, the setup cannot be reset if journal entries beyond the opening balance exist	Application-level check on reset endpoint

16.6 Integration Rules

Rule ID	Rule	Enforcement
BR-INT-001	One SMS account can be mapped to one external code per external system type	Unique index on (system_type, account_id, org_id)
BR-INT-002	Only Level 4 (leaf) accounts can be mapped to external systems	Application validation on mapping create
BR-INT-003	Mapped accounts must be active (is_active = true)	Application validation
BR-INT-004	Sync direction 'None' means the mapping is stored for reference only — no automated sync	Application logic
BR-INT-005	connection_config_json must never contain plaintext credentials. Secrets are stored in the system vault and referenced by key	Application logic + code review enforcement

17. Test Scenarios

17.1 CoA Template Tests

Test ID	Scenario	Expected Result
T-TPL-01	List templates filtered by country_code = 'USA'	Returns US_GENERAL, US_RETAIL, US_MANUFACTURING, US_SERVICES
T-TPL-02	Download US_GENERAL template as CSV	Returns valid CSV with all 5 root types, correct parent_code references
T-TPL-03	Download US_GENERAL template as JSON	Returns valid JSON tree with nested children structure
T-TPL-04	Attempt to modify a system template (org_id = NULL)	Returns 403 Forbidden
T-TPL-05	Preview template with 70 accounts	Returns flat list with 70 items, correct level and root_type

17.2 CoA Upload & Validation Tests

Test ID	Scenario	Expected Result
T-UPL-01	Upload valid CSV with 50 accounts	CoA created with 50 accounts, status → AccountsCreated
T-UPL-02	Upload CSV missing Level 1 "Equity" root	Validation error: "Missing root type: Equity"
T-UPL-03	Upload CSV with duplicate account code "1111"	Validation error: "Duplicate account code: 1111"
T-UPL-04	Upload CSV with Level 5 account	Validation error: "Invalid level 5 for account..."
T-UPL-05	Upload CSV with orphan parent reference	Validation error: "Account X references parent Y which is not found..."
T-UPL-06	Upload CSV with 2001 accounts	Validation error: "Maximum allowed is 2000"
T-UPL-07	Upload valid JSON tree file	CoA created with correct hierarchy, materialized_path auto-computed
T-UPL-08	Upload invalid JSON (malformed)	Validation error: "Invalid file format"
T-UPL-09	Upload CSV where Level 4 is_group = true	Validation error: "Level 4 must not be marked as a group"
T-UPL-10	Upload CSV after journal entries exist	Validation error: "Cannot replace after journal entries posted"

17.3 CoA Hierarchy Tests

Test ID	Scenario	Expected Result
T-COA-01	Apply US_GENERAL template	All template accounts created with correct parent_account_id, materialized_path, root_type
T-COA-02	Add Level 4 account under Level 3 parent	Account created, materialized_path = parent.path + "/new_code"
T-COA-03	Attempt to add Level 3 account under Level 1 parent	Validation error: parent must be Level 2
T-COA-04	Attempt to add account with root_type different from parent's root_type	Validation error: root_type must match root ancestor
T-COA-05	Delete Level 4 account with no transactions	Account soft-deleted (is_active = false)
T-COA-06	Attempt to delete Level 4 account with has_transactions = true	Error: "Cannot delete account with posted transactions"
T-COA-07	Attempt to delete Level 3 group with children	Error: "Cannot delete account with children"
T-COA-08	Attempt to delete system account (3900)	Error: "Cannot delete system account"
T-COA-09	Export tenant CoA as CSV	Valid CSV with all accounts, correct parent_code references
T-COA-10	Query subtree under "11 — Current Assets" using materialized_path LIKE	Returns all accounts with path starting with '/1/11/'
T-COA-11	Multi-tenant isolation: Tenant A's accounts not visible to Tenant B	Tenant B query returns 0 results for Tenant A's accounts

17.4 Fiscal Period Tests

Test ID	Scenario	Expected Result
T-FP-01	Create fiscal year Apr 2026 – Mar 2027	12 monthly periods + 1 adjustment period auto-created
T-FP-02	Attempt overlapping fiscal year	Validation error: "Fiscal year dates overlap with existing year"
T-FP-03	Lock fiscal period	Status changes to Locked, postings blocked, reopen possible
T-FP-04	Close fiscal period	Status changes to Closed, postings blocked, reopen blocked
T-FP-05	Attempt to close year with open periods	Error: "All periods must be closed first"

17.5 Opening Balance Tests

Test ID	Scenario	Expected Result
T-OB-01	Enter balanced opening balances (debits = credits)	All entries saved, balanced flag = true
T-OB-02	Enter unbalanced opening balances	Difference calculated, user warned, difference posted to 3900 on confirmation
T-OB-03	Post opening balances	Entry status → Posted, ChartOfAccounts.opening_balance_debit/credit updated, has_transactions = true
T-OB-04	Attempt to modify posted opening balance	Error: "Posted balances cannot be modified"
T-OB-05	Skip opening balances for new company	All balances remain zero, opening_balances_entered = true, CoA setup proceeds
T-OB-06	Attempt to enter opening balance on Level 3 (group) account	Validation error: "Only Level 4 accounts accept opening balances"
T-OB-07	Enter both debit and credit > 0 for same account	Validation error: "Cannot have both debit and credit amounts"

17.6 Setup Status & Module Gate Tests

Test ID	Scenario	Expected Result
T-SS-01	New tenant activates Finance module	CoaSetupStatuses row created with status = NotStarted
T-SS-02	Apply template → status advances to AccountsCreated	Status = AccountsCreated, template_id set
T-SS-03	Create fiscal year → status advances to FiscalYearSet	Status = FiscalYearSet, fiscal_year_configured = true
T-SS-04	Assign all required defaults → status advances to DefaultsSet	Status = DefaultsSet, defaults_configured = true
T-SS-05	Post opening balances → status advances to BalancesEntered	Status = BalancesEntered, opening_balances_entered = true
T-SS-06	Complete setup → status = Complete, setup_completed_at set	Status = Complete, financial modules unlocked
T-SS-07	Access /api/finance/invoices before setup complete	403 FINANCE_SETUP_REQUIRED
T-SS-08	Access /api/finance/setup/status before setup complete	200 OK (setup endpoints are exempt from gate)
T-SS-09	Reset setup after only opening balances exist	Status returns to NotStarted, all setup data cleared
T-SS-10	Attempt reset after regular journal entries exist	Error: "Cannot reset after journal entries posted"

17.7 External Mapping Tests

Test ID	Scenario	Expected Result
T-EXT-01	Create QB Online mapping for account 1111	Mapping created with system_type = QUICKBOOKS_ONLINE
T-EXT-02	Attempt duplicate mapping (same system + same account)	Unique constraint violation
T-EXT-03	Import 20 mappings from CSV	20 mapping rows created
T-EXT-04	Map to group account (Level 3)	Validation error: "Only Level 4 accounts can be mapped"
T-EXT-05	Map to inactive account	Validation error: "Mapped account must be active"

---

18. Development Phases

Phase 1 — Foundation (Estimated: 8 dev days)

Task	Description	Days
1A	Create `finance` schema, update Module Catalog (M1)	0.5
1B	Create CoaTemplates entity, migration M2	1
1C	Create CoaTemplateItems entity, migration M4	1
1D	Seed GLOBAL_GENERAL and US_GENERAL templates (M3, M5, M6)	2
1E	Template API endpoints (list, detail, download CSV/JSON, preview)	2
1F	Template selection UI component	1.5

Phase 2 — Chart of Accounts (Estimated: 10 dev days)

Task	Description	Days
2A	Create ChartOfAccounts entity, migration M7	1.5
2B	Apply-template service (copy template items → tenant accounts, compute materialized_path)	2
2C	Upload service (CSV parser, JSON parser, validation engine — 13 rules)	3
2D	CoA CRUD API endpoints (tree, flat, create, update, delete, export)	2
2E	CoA tree view UI (collapsible hierarchy, search, inline balance display)	1.5

Phase 3 — Fiscal Periods & Defaults (Estimated: 5 dev days)

Task	Description	Days
3A	Create FiscalYears and FiscalPeriods entities, migration M8	1
3B	Fiscal year creation with auto-period generation	1
3C	Create AccountingDefaults entity, migration M9, seed default keys	1
3D	Defaults CRUD API and validation service	1
3E	Fiscal year and defaults UI panels	1

Phase 4 — Opening Balances & Setup Gate (Estimated: 6 dev days)

Task	Description	Days
4A	Create OpeningBalanceEntries entity, migration M11	1
4B	Opening balance batch entry API and posting logic	2
4C	Create CoaSetupStatuses entity, migration M10, state machine	1
4D	FinanceSetupGateMiddleware + React route guard	0.5
4E	Opening balance entry UI (grouped table, difference calculator, posting confirmation)	1.5

Phase 5 — Wizard, Guide & Integration Hooks (Estimated: 5 dev days)

Task	Description	Days
5A	CoA Setup Wizard (6-step flow with progress tracking)	2
5B	Contextual help system (expandable panels, per-step content)	1
5C	Create ExternalSystemMappings entity, migration M12	0.5
5D	External mapping CRUD API and CSV import	1
5E	External mappings UI panel	0.5

Phase 6 — Testing & Polish (Estimated: 4 dev days)

Task	Description	Days
6A	Unit tests for validation engine (upload rules, hierarchy checks)	1.5
6B	Integration tests (template apply → CoA → opening balances → setup complete)	1.5
6C	Multi-tenant isolation tests	0.5
6D	UI polish, responsive layout, dark mode verification	0.5

Total Estimated Effort: ~38 dev days

Appendix A — Account Type Reference

This appendix lists the `account_type` sub-classifications available for Level 4 accounts and their valid root types.

account_type	Valid Root Types	Description
Cash	Asset	Physical cash, petty cash
Bank	Asset	Bank accounts (checking, savings)
Receivable	Asset	Amounts owed by customers
OtherCurrentAsset	Asset	Prepaid expenses, deposits, other current assets
FixedAsset	Asset	Property, plant, equipment
AccumulatedDepreciation	Asset	Contra-asset account for depreciation
Payable	Liability	Amounts owed to suppliers
OtherCurrentLiability	Liability	Accrued expenses, short-term borrowings
LongTermLiability	Liability	Loans, bonds, long-term debt
Equity	Equity	Owner's capital, retained earnings, reserves
Revenue	Revenue	Sales income, service income
OtherIncome	Revenue	Interest income, discounts received, gains
CostOfGoodsSold	Expense	Direct costs of goods sold
Expense	Expense	Operating expenses (salaries, rent, utilities)
OtherExpense	Expense	Non-operating expenses, losses
Tax	Asset or Liability	Tax receivable (input VAT) or tax payable (output VAT)
Depreciation	Expense	Depreciation expense

Appendix B — Domain Events

Event Name	Trigger	Payload
CoaTemplateAppliedEvent	Template applied to create tenant CoA	OrgId, TemplateId, AccountCount
CoaUploadedEvent	Tenant uploaded CSV/JSON to create CoA	OrgId, FileName, AccountCount, Format
ChartOfAccountCreatedEvent	New account added to tenant CoA	OrgId, AccountId, AccountCode, Level, RootType
ChartOfAccountUpdatedEvent	Account modified	OrgId, AccountId, ChangedFields
ChartOfAccountDeletedEvent	Account soft-deleted	OrgId, AccountId, AccountCode
FiscalYearCreatedEvent	New fiscal year created	OrgId, FiscalYearId, StartDate, EndDate
FiscalPeriodStatusChangedEvent	Period opened/locked/closed	OrgId, PeriodId, OldStatus, NewStatus
AccountingDefaultAssignedEvent	Default account assigned	OrgId, DefaultKey, AccountId
OpeningBalancePostedEvent	Opening balances finalized	OrgId, TotalDebit, TotalCredit, Difference, ConversionDate
CoaSetupStatusChangedEvent	Setup status transition	OrgId, OldStatus, NewStatus
CoaSetupCompletedEvent	Setup finalized	OrgId, CompletedAt, CompletedByUserId, AccountCount
ExternalMappingCreatedEvent	Account mapped to external system	OrgId, MappingId, SystemType, AccountId, ExternalCode

These events enable future cross-module integrations (dashboard refresh, notifications, audit logs) without tight coupling.

Appendix C — CSV/JSON Schema Specification

C.1 CSV Upload Schema

The CSV upload file must have the following header row (column order is significant):

```
account_code,account_name,parent_code,level,root_type,is_group,account_type,description
```

Column	Required	Type	Notes
account_code	Yes	string	Unique within file. Digits recommended (e.g., "1111")
account_name	Yes	string	Max 200 characters
parent_code	No	string	Account code of the parent. Empty for Level 1 roots
level	Yes	integer	1, 2, 3, or 4
root_type	Yes	string	Asset, Liability, Equity, Revenue, Expense
is_group	Yes	boolean	true or false (text)
account_type	No	string	See Appendix A for valid values. Empty for group accounts
description	No	string	Max 500 characters

C.2 JSON Upload Schema

```json
```

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "SMS Chart of Accounts Upload",
  "type": "object",
  "required": ["accounts"],
  "properties": {
    "template_name": { "type": "string" },
    "country_code": { "type": "string", "maxLength": 3 },
    "currency_code": { "type": "string", "maxLength": 3 },
    "accounts": {
      "type": "array",
      "items": { "$ref": "#/$defs/account" }
    }
  },
  "$defs": {
    "account": {
      "type": "object",
      "required": ["code", "name", "root_type", "level", "is_group"],
      "properties": {
        "code": { "type": "string", "maxLength": 20 },
        "name": { "type": "string", "maxLength": 200 },
        "root_type": { "type": "string", "enum": ["Asset", "Liability", "Equity", "Revenue",
        "level": { "type": "integer", "minimum": 1, "maximum": 4 },
        "is_group": { "type": "boolean" },
        "account_type": { "type": "string" },
        "description": { "type": "string", "maxLength": 500 },
        "children": {
          "type": "array",
          "items": { "$ref": "#/$defs/account" }
        }
      }
    }
  }
}
```

End of Addendum 42